IC-253 (DSA) Assignment 3 Report

Aryan Sisodiya

April 11, 2026

1 Task 1: AVL Tree-Based File System Directory

1.1 Problem Description

This task simulates a file system directory using an AVL Tree. Each node stores a filename (string) and file size (integer). The tree supports insertion, deletion, searching, finding minimum and maximum filenames, and in-order display with balance factors.

1.2 Approach

Each node in the AVL tree stores:

- Filename (`fname`)
- File size (`fsize`)
- Left and right child pointers
- Height of the node for balancing

The tree is kept balanced using standard AVL rotations. During insertion and deletion, the balance factor is checked and one of the four rotations is applied when needed: LL, RR, LR, or RL. The implementation also prints the triggered rotation type, which helps verify the balancing logic during testing.

1.3 Filename Policy

The filename policy is enforced before insertion and is based on the checks implemented in the code:

- Only alphanumeric characters and the special characters `_`, `.`, and `-` are allowed.
- Filename length must be at least 1 and strictly less than 20 characters.
- A filename cannot start or end with a special character.
- Repeating special characters such as `..`, `__`, or `--` are not allowed.
- Invalid filenames are rejected immediately, and a specific reason is printed.

The ordering rule used by the AVL tree comparator is:

1. Special characters have the highest priority.
2. Digits have lower priority than special characters, but higher priority than alphabets.
3. Alphabets have the lowest priority.

4. If two characters have the same priority, standard ASCII ordering is used.
5. If a common prefix still remains, the longer filename is given higher priority.

1.4 Functions and Implementation

validateFilename(name)
Checks whether the filename obeys the policy above.

insert(name, size)
Inserts a valid file into the AVL tree and rebalances the tree if required.

delete_(name)
Deletes a file by name, handles all three deletion cases, and rebalances the tree.

search(name)
Searches for a file using the custom comparator ordering.

findMin() / findMax()
Returns the lexicographically smallest / largest filename under the defined ordering.

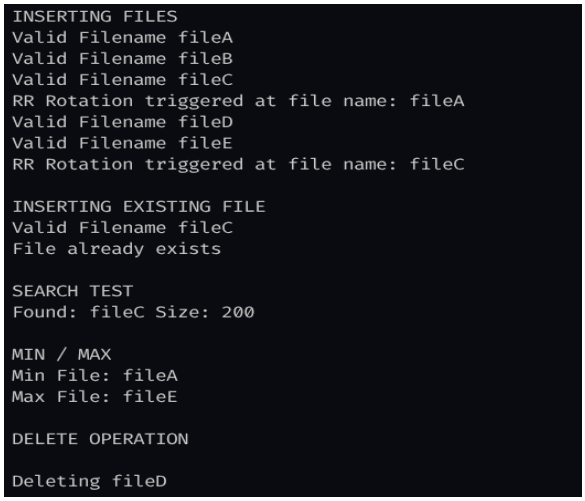
display()
Prints the tree using in-order traversal and shows the balance factor of each node.

1.5 Time and Space Complexity Analysis

Operation	Time Complexity	Space Complexity
validateFilename()	$O(L)$	$O(1)$
insert()	$O(\log n)$	$O(\log n)$
delete_()	$O(\log n)$	$O(\log n)$
search()	$O(\log n)$	$O(\log n)$
findMin() / findMax()	$O(\log n)$	$O(1)$
display()	$O(n)$	$O(\log n)$

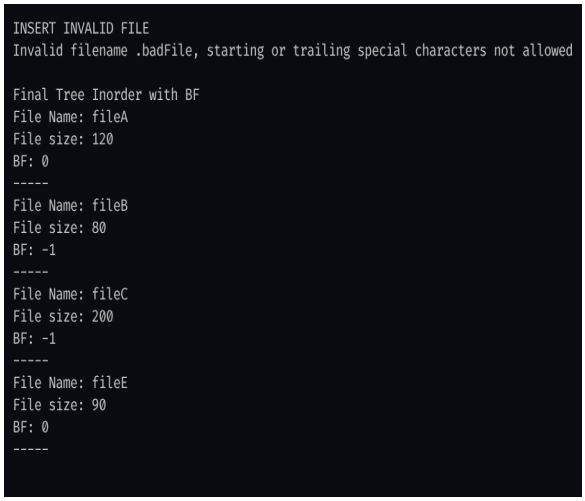
Here, L is the length of the filename and n is the number of files stored in the tree. The recursive operations use call stack space proportional to the height of the AVL tree.

1.6 Test Case Demonstrations



```
INSERTING FILES
Valid Filename fileA
Valid Filename fileB
Valid Filename fileC
RR Rotation triggered at file name: fileA
Valid Filename fileD
Valid Filename fileE
RR Rotation triggered at file name: fileC

INSERTING EXISTING FILE
Valid Filename fileC
File already exists

SEARCH TEST
Found: fileC Size: 200

MIN / MAX
Min File: fileA
Max File: fileE

DELETE OPERATION

Deleting fileD
```

(a) Output Part 1



```
INSERT INVALID FILE
Invalid filename .badFile, starting or trailing special characters not allowed

Final Tree Inorder with BF
File Name: fileA
File size: 120
BF: 0
-----
File Name: fileB
File size: 80
BF: -1
-----
File Name: fileC
File size: 200
BF: -1
-----
File Name: fileE
File size: 90
BF: 0
-----
```

(b) Output Part 2

Figure 1: Task 1 – Test Case 1 Output

```
TEST CASE 2
INSERTING FILES (WITH MIXED PRIORITY)
Valid Filename alpha
Valid Filename Alpha
Valid Filename alpha_1
Valid Filename alpha-2
Valid Filename alpha.3
LR Rotation triggered at file name: alpha_1

INSERT INVALID FILES
Invalid filename _startBad, starting or trailing special characters not allowed
Invalid filename endBad_, starting or trailing special characters not allowed
Invalid filename double__underscore, Repeating special characters not allowed

SEARCH TEST (EXISTING + NON-EXISTING)
Found: alpha_1 Size: 150
ghostFile not found

MIN / MAX
Min File: Alpha
Max File: alpha_1
```

(a) Output Part 1

```
DELETE TEST (EXISTING + NON-EXISTING)
RR Rotation triggered at file name: Alpha
Given file not present in the directory

TREE AFTER DELETIONS
File Name: Alpha
File size: 200
BF: -1
-----
File Name: alpha-2
File size: 90
BF: 0
-----
File Name: alpha.3
File size: 60
BF: 1
-----
File Name: alpha_1
File size: 150
BF: 0
-----
```

(b) Output Part 2

Figure 2: Task 1 – Test Case 2 Output

2 Task 2: Binary Expression Tree with Auto-Detection

2.1 Problem Description

This task builds a binary expression tree from an input expression string. The program first detects whether the expression is in infix, prefix, or postfix form, and then constructs the tree accordingly. Each internal node stores an operator and each leaf node stores an operand.

2.2 Approach

The implementation works character by character, so multi-digit numbers are treated as separate digits. The tree supports:

- automatic format detection,
- expression validation,
- tree construction from infix, prefix, or postfix form,
- evaluation using a variable map,
- conversion back to infix, prefix, and postfix forms,
- node counting.

The code uses stack-based tree construction for prefix and postfix forms, while infix expressions are first converted to postfix and then built as a tree. Evaluation uses `double` so division is true division instead of integer truncation.

2.3 Expression Validation Policy

The expression is treated strictly as a *character-wise* expression tree: each operand is a single alphanumeric character and each operator is one of `+`, `-`, `*`, or `/`. Multi-digit numbers are therefore not parsed as single tokens; instead, each digit is interpreted as a separate operand. This is consistent with the tree-building and evaluation logic in the code using the variable Map.

The validation policy enforced by the program is:

- **Allowed operands:** any single letter or digit (`isalnum` characters).

- **Allowed operators:** `+`, `-`, `*`, and `/`.
- **Allowed brackets:** only `(` and `)` are permitted, and only for infix expressions.
- **Unknown symbols:** any character outside the allowed set is rejected immediately.
- **Balanced parentheses:** infix expressions must have properly balanced parentheses.
- **Structural validity:**
  - In **infix**, operands cannot appear consecutively, operators cannot appear consecutively, an operand cannot be immediately followed by `(`, and `)` cannot be immediately followed by an operand.
  - In **prefix** and **postfix**, the expression must satisfy the stack-balance rule: every operator must combine exactly two valid subexpressions, and the final expression must reduce to exactly one tree.
- **Rejection policy:** invalid expressions are rejected entirely, and the program prints a specific reason such as invalid character, consecutive operators, unbalanced parentheses, malformed prefix/postfix structure, or division-related evaluation errors.

2.4 Functions and Implementation

detectFormat(expression)
Identifies whether the input expression is infix, prefix, postfix, or unrecognized.

validateExpression(expression)
Checks whether the expression follows the selected validation policy and reports the reason if invalid.

buildTree(expression)
Constructs the binary expression tree only after successful validation.

evaluate(varMap)
Evaluates the tree using a mapping from variable names to values.

toInfix()
Returns the infix representation with proper parentheses.

toPrefix()
Returns the prefix representation of the tree.

toPostfix()
Returns the postfix representation of the tree.

countNodes()
Returns the total number of nodes, internal nodes, and leaf nodes.

2.5 Time and Space Complexity Analysis

Operation	Time Complexity	Space Complexity
detectFormat()	$O(n)$	$O(1)$
validateExpression()	$O(n)$	$O(1)$
buildTree()	$O(n)$	$O(n)$
evaluate()	$O(n)$	$O(h)$
toInfix() / toPrefix() / toPostfix()	$O(n)$	$O(h)$
countNodes()	$O(n)$	$O(h)$

Here, n is the length of the expression and h is the height of the expression tree.

2.6 Test Case Demonstrations

```
Testing Expression: ab+c*

EXPRESSION VALIDATION
Expression is VALID

FORMAT DETECTION
Detected Format: POSTFIX

BUILDING TREE
Tree constructed successfully

REPRESENTATIONS
Infix   : ((a+b)*c)
Prefix  : *+abc
Postfix : ab+c*

EVALUATION
Evaluated Result at a=2,b=3,c=4,d=5: 20

NODE COUNTS
Total Nodes   : 5
Internal Nodes: 2
Leaf Nodes    : 3
```

(a) Test Case 1

```
Testing Expression: (a+b)*(c-d)

EXPRESSION VALIDATION
Expression is VALID

FORMAT DETECTION
Detected Format: INFIX

REPRESENTATIONS
Infix   : ((a+b)*(c-d))
Prefix  : *+ab-cd
Postfix : ab+cd-*

EVALUATION
Evaluated Result: 49

NODE COUNTS
Total Nodes   : 7
Internal Nodes: 3
Leaf Nodes    : 4
```

(b) Test Case 2

```
Testing Expression: *+ab-cd

EXPRESSION VALIDATION
Expression is VALID
Detected Format: PREFIX

Prefix  : *+ab-cd
Infix   : ((a+b)*(c-d))
Postfix : ab+cd-*
Evaluated Result: -3

Testing Expression: a++b

EXPRESSION VALIDATION
Invalid number of operators/operand
Expression is INVALID
Detected Format: UNRECOGNISED
Unrecognised expression
Invalid expression correctly rejected
```

(c) Test Case 3

```
Testing Expression: ((a+b)*(c+d)-(e/f+g*h))

EXPRESSION VALIDATION
Expression is VALID
Detected Format: INFIX

REPRESENTATIONS
Infix   : (((a+b)*(c+d))-((e/f)+(g*h)))
Prefix  : -*+ab+cd+/ef*gh
Postfix : ab+cd+*ef/gh*+-

EVALUATION
Evaluated Result: 23

NODE COUNTS
Total Nodes   : 15
Internal Nodes: 7
Leaf Nodes    : 8
```

(d) Test Case 4

Figure 3: Task 2 – Test Case Outputs